

TP3-2

December 8, 2025

1 TP 3 - Grupo 7

André Filipe Dourado Pinheiro - A108473

Tiago Silva Costa - A108657

1.1 Problema 2 - Algoritmo estendido de Euclides (CFA)

Na continuação do problema 1 pretende-se provar a correção do programa à apresentado.

1. Identifique um CFA que representa o programa. Nomeadamente identifique
 1. os locais e os transformadores de predicados “weakest pre-condition” que descrevem as transições de estado em cada local.
 2. as guardas que determinam as transições de local
 3. os locais que representam as situações de erro e os que representam a terminação com sucesso.
2. Usando k -indução verifique que $\phi(a, b, r, s, t) \equiv a * s + b * t = r$ é invariante.
3. Usando a metodologia dos “look-aheads” verifique que o programa termina sempre.

1.2 Modelação

Com o objetivo de resolver o problema proposto, optou-se pela utilização do módulo `pysmt.shortcuts`, que oferece diversos recursos voltados ao uso de SMT Solvers. Além disso, os tipos de dados específicos do Solver são importados a partir do módulo `pysmt.typing`.

```
[45]: from pysmt.shortcuts import *  
      from pysmt.typing import BOOL
```

1.3 Criação do CFA

O algoritmo foi modelado por um CFA sumariamente descrito no seguinte diagrama:

1. Todos os locais estão identificados com um nome
2. A inicialização é descrita por uma transição `havoc`, que atribui às variáveis a, b valores arbitrários, seguida da seguinte guarda

$$(r = a) \wedge (r' = b) \wedge (s = 1) \wedge (s' = 0) \wedge (t = 0) \wedge (t' = 1)$$

que deixa passar apenas valores apropriados para o nosso problema.

Pode-se assumir que o predicado dos estados iniciais é

$$I \equiv \text{True}$$

e que o 1º estado de acessibilidade $\text{sp}(I)$ é determinado por este comando `havoc` seguido pela guarda referida. Por isso

$$\text{sp}(I) = \bigvee a, b \cdot \underbrace{(r = a) \wedge (r' = b) \wedge (s = 1) \wedge (s' = 0) \wedge (t = 0) \wedge (t' = 1)}_{\text{Guarda } \mathbf{g}_{init}}$$

Todos os restantes locais vão ser representados por pré-condições mais fracas.

Podemos associar um predicado a cada um dos locais do sistema de acordo com os seguintes princípios. - Ao local `system` associamos a condição de segurança atrás referida - Aos restantes locais com excepção de `stop` e `error` associamos a pré-condição mais fraca. - Aos locais `stop` e `error` associamos predicados constantes que vão depender do tipo de problema que vamos tentar modelar.

Ignorando por momentos a semântica pode-se escrever num sistema de equações

- $\text{system} \equiv \bigwedge_{a,b} ((r = a) \wedge (r' = b) \wedge \dots \rightarrow \text{do})$
- $\text{do} \equiv (\underbrace{r' = 0}_{\text{Guarda } \mathbf{g}_{stop}} \rightarrow \text{stop}) \wedge (\underbrace{r' \neq 0}_{\text{Guarda } \mathbf{g}_{step}} \rightarrow \text{step})$
- $\text{step} \equiv \text{WP}_{\text{step}} \left((\underbrace{\text{overflow}}_{\text{Guarda } \mathbf{g}_{error}} \rightarrow \text{error}) \wedge (\underbrace{\neg \text{overflow}}_{\text{Guarda } \mathbf{g}_{cont}} \rightarrow \text{step}) \right)$

Em que

$$\text{WP}_{\text{step}}(\mathbf{Q}) = \mathbf{Q}[r/r', r'/(r - q \cdot r'), s/s', s'/(s - q \cdot s'), t/t', t'/(t - q \cdot t')]$$

Para um dado predicado $\mathbf{Q}$ e para as seguintes guardas:

- $\mathbf{g}_{init} \equiv (r = a) \wedge (r' = b) \wedge (s = 1) \wedge (s' = 0) \wedge (t = 0) \wedge (t' = 1)$
- $\mathbf{g}_{stop} \equiv r' = 0$
- $\mathbf{g}_{step} \equiv r' \neq 0$
- $\mathbf{g}_{error} \equiv \text{overflow}$
- $\mathbf{g}_{cont} \equiv \neg \text{overflow}$

Por fim, como vamos querer verificar um invariante e também verificar se o programa sempre termina, vamos definir os seguintes predicados

- $\text{stop} \equiv \text{True}$
- $\text{error} \equiv \text{False}$

1.4 Utilização de k -indução

Queremos agora utilizar k -indução para verificar que $\phi(a, b, r, s, t) \equiv a * s + b * t = r$ é invariante.

Para isso, vamos definir o SFOTS que definimos no problema 1 mas com ligeiras modificações. Em primeiro lugar, vamos definir com a syntax do `pysmt` e vamos utilizar um valor de $N = 6$.

[46]:

```
N = 6
BV_0 = BV(0, N)
```

```

BV_1 = BV(1, N)

vars_names = ['r', 's', 't', 'rp', 'sp', 'tp', 'pc']

def declare(i):
    state = {}
    for v in vars_names:
        state[v] = Symbol(f"{v}_{i}", BVType(N))
    return state

def init_ab(state,a,b):
    return And(
        Equals(state['r'], BV(a,N)),
        Equals(state['rp'], BV(b,N)),

        Equals(state['s'], BV_1),
        Equals(state['sp'], BV_0),
        Equals(state['t'], BV_0),
        Equals(state['tp'], BV_1),

        Equals(state['pc'], BV_1)
    )

def trans1(curr, nxt):
    r, rp = curr['r'], curr['rp']
    s, sp = curr['s'], curr['sp']
    t, tp = curr['t'], curr['tp']
    pc = curr['pc']

    r_, rp_ = nxt['r'], nxt['rp']
    s_, sp_ = nxt['s'], nxt['sp']
    t_, tp_ = nxt['t'], nxt['tp']
    pc_ = nxt['pc']

    is_running = And(Equals(pc, BV_1), Not(Equals(rp, BV_0)))
    q = BVUDiv(r, rp)

    return And(
        # Atualização do PC
        Equals(pc_, Ite(is_running, BV_1, BV_0)),

        # Algoritmo de Euclides
        Equals(r_, Ite(is_running, rp, r)),
        Equals(rp_, Ite(is_running, BVURem(r, rp), rp)),

        Equals(s_, Ite(is_running, sp, s)),
        Equals(sp_, Ite(is_running, BVSub(s, BVMul(q, sp)), sp)),
    )

```

```

    Equals(t_, Ite(is_running, tp, t)),
    Equals(tp_, Ite(is_running, BVSub(t, BVMul(q, tp)), tp))
)

```

Podemos agora definir o invariante que queremos verificar, que no caso é $\phi(a, b, r, s, t) \equiv a * s + b * t = r$. É importante verificar este mesmo invariante para as variáveis r', s', t' , ou seja, verificar $\phi(a, b, r', s', t')$.

Para isso, vamos definir a função `inv`.

```

[47]: def inv(s,a,b):
    prop_curr = Equals(BVAdd(BVMul(BV(a,N), s['s']), BVMul(BV(b,N), s['t'])),
    ↪s['r'])
    prop_next = Equals(BVAdd(BVMul(BV(a,N), s['sp']), BVMul(BV(b,N), s['tp'])),
    ↪s['rp'])

    return And(prop_curr, prop_next)

```

Por fim, como descrito nas aulas teóricas e práticas, o seguinte algoritmo expande o conceito da indução para passos de transição, testando também estados iniciais.

Portanto, utilizando a função `kinduction_always` (implementada nas aulas práticas), podemos utilizar k -indução.

```

[48]: def kinduction_always(declare,init,trans,inv,k,a,b):

    with Solver(name="z3") as solver:

        states = [declare(i) for i in range(k)]

        base = [init(states[0],a,b)]

        for i in range(k-1):
            base.append(trans(states[i],states[i+1]))

        caso_base = And( And(base), Or((Not(inv(s,a,b)) for s in states)))

        solver.add_assertion(caso_base)
        if solver.solve():
            print("Caso base não respeita o invariante.")
            print("CE: ", solver.get_model())
            return False
        solver.reset_assertions()

        # Passo Indutivo
        states = [declare(i) for i in range(k+1)]
        inductive = []

```

```

for i in range(k):
    inductive.append(trans(states[i],states[i+1]))
    inductive.append(And([inv(states[i],a,b) for i in range(k)]))
    inductive.append(Not(inv(states[k],a,b)))

step = And(inductive)
solver.add_assertion(step)
if solver.solve():
    print("Passo indutivo não respeita o invariante.")
    print("CE: ", solver.get_model())
    return False

print("Invariante verificado por k-indução.")
return True

```

Como os traços são limitados, basta testar k -indução para $k = \left\lfloor \frac{\ln(2^N-1)}{\ln(\phi)} \right\rfloor + 1$, já que é possível mostrar matematicamente que a *upper_bound* do número máximo de passos será

$$N_{max} < \frac{\ln(b)}{\ln(\phi)} + 1$$

onde $b < a$. Portanto, para calcular o comprimento máximo do traço permitido pela arquitetura de N bits, substituímos o menor input b pelo valor máximo possível nos N bits, $b = 2^N - 1$.

Sendo assim, $\left\lfloor \frac{\ln(2^{16}-1)}{\ln(1.618)} \right\rfloor + 1 \approx 25$

```
[49]: kinduction_always(declare,init_ab,trans1,inv,25,50,49)
```

Invariante verificado por k-indução.

[49]: True

```
[50]: kinduction_always(declare,init_ab,trans1,inv,25,13,49)
```

Invariante verificado por k-indução.

[50]: True

1.5 Metodologia dos *look-aheads*

Pretende-se agora utilizar a metodologia dos “look-aheads” para verificar que o programa termina sempre.

Para isso, vamos definir o seguinte invariante que é dado pela expressão

$$r'$$

na função `variante(state)`.

```
[51]: def variante(state):
      return state['rp']
```

Além disso, vamos criar a função auxiliar `init1(state)` que será responsável por definir o estado inicial sem definir *a* e *b* fixos.

```
[52]: def init1(state):
      return And(
          BVUGT(state['r'], BV_0),
          BVUGT(state['rp'], BV_0),
          Equals(state['s'], BV_1),
          Equals(state['sp'], BV_0),
          Equals(state['t'], BV_0),
          Equals(state['tp'], BV_1),

          Equals(state['pc'], BV_1)
      )
```

Finalmente, define-se a função `k_lookahead` que foi implementada nas aulas práticas (em particular na ficha 8), que será responsável por verificar a terminação por *Look-Aheads*.

```
[53]: def k_lookahead(declare, init, trans, var, k):
      with Solver(name="z3") as s:
          states = [declare(i) for i in range(k+1)]

          s.add_assertion(init(states[0]))

          transitions = [trans(states[i], states[i+1]) for i in range(k)]

          all_trans = And(transitions)

          decreases = Or(
              var(states[-1]) < var(states[0]),
              Equals(var(states[-1]), BV_0)
          )

          s.add_assertion(And(all_trans, Not(decreases)))

          if s.solve():
              print(f"Existe uma seq. de {k} passos onde o variante não decresce.
↪")

              print("CE: ", s.get_model())
              return False

          print(f"0 variante decresce a cada {k} passos, ou atinge 0.")
```

Por fim, podemos testar com alguns testes

[54]: `k_lookahead(declare,init1,trans1,variante,8)`

0 variante decresce a cada 8 passos, ou atinge 0.

[55]: `k_lookahead(declare,init1,trans1,variante,67)`

0 variante decresce a cada 67 passos, ou atinge 0.

Portanto, podemos concluir que **o programa sempre termina**.